

Python Operators – Applied Learning Case Studies

Case Study 1 — Student Result Dashboard

1. Problem Statement

A college wants a Python application to calculate a student's academic performance. The student enters the name, student ID, internal marks, assignment marks, mid-term marks, end-semester marks, and attendance percentage. The application should calculate total marks, percentage, grade, and examination eligibility.

Test the program using at least two normal cases and one edge case.

Expected Dashboard

STUDENT RESULT DASHBOARD	
Student Name	: _____
Student ID	: _____
Internal Marks	: _____ / 20
Assignment Marks	: _____ / 15
Mid-Term Marks	: _____ / 25
End-Semester	: _____ / 50
Total Marks	: _____
Percentage	: _____ %
Grade	: _____
Attendance	: _____ %
Examination Status	: ELIGIBLE / NOT ELIGIBLE
Final Result	: PASS / FAIL

Case Study 2 — Online Shopping Dashboard

1. Problem Statement

- An online shopping company wants to automatically generate a customer's bill. The customer enters product name, quantity, price, and coupon code. If the subtotal is at least ₹5,000, provide a 10% discount. A valid coupon provides an additional discount. If the final amount is at least ₹3,000, delivery is free. **Test the program using at least two normal cases and one edge case.**

2. Expected Dashboard

ONLINE SHOPPING DASHBOARD	
Product	: _____
Quantity	: _____
Price per Item	: ₹ _____
Coupon Code	: _____
Subtotal	: ₹ _____
Discount	: ₹ _____
Coupon Discount	: ₹ _____
Delivery Charge	: ₹ _____
Final Amount	: ₹ _____
Coupon Status	: VALID / INVALID
Delivery Status	: FREE / CHARGED
Order Status	: CONFIRMED / REJECTED

3. Questions for Students

- How will you calculate the subtotal?
- How will you determine whether the customer gets a discount?
- How can you check whether a coupon exists in the valid-coupon list?
- How will you decide whether delivery is free?

Hints

- Subtotal = price * quantity.
- Use ≥ 5000 to check discount eligibility.
- Store valid coupons in a list and use the in operator.

Case Study 3 — University Examination Eligibility Dashboard

1. Problem Statement

A university wants to automatically determine whether a student is eligible for the semester examination. A student is eligible when attendance is at least 75%, internal marks are at least 40%, the assignment has been submitted, and fees have been paid.

2. Expected Dashboard

EXAMINATION ELIGIBILITY DASHBOARD	
Student Name	: _____
Attendance	: _____ %
Internal Marks	: _____ %
Assignment	: SUBMITTED / NOT SUBMITTED
Fees	: PAID / NOT PAID
Attendance	: ✓ / X
Academic Criteria	: ✓ / X
Assignment	: ✓ / X
Fee Status	: ✓ / X
EXAMINATION STATUS	
ELIGIBLE / NOT ELIGIBLE	

3. Questions for Students

5. How will you represent fees_paid?
6. How will you represent assignment_submitted?
7. How will you check all four conditions together?
8. What happens if one condition is false?
9. Can you use a ternary operator for the final status?

4. Hints

- Use Boolean variables such as fees_paid = True.
- Use attendance >= 75 and internal_marks >= 40.
- If all requirements are compulsory, think about and.
- Use the final Boolean expression to determine eligibility.

Case Study 4 — ATM Transaction Dashboard

1. Problem Statement

Design a simple ATM transaction system. The customer can deposit money, withdraw money, or check the balance. Withdrawal is allowed only when the amount is greater than zero, does not exceed the withdrawal limit, and sufficient balance is available.

2. Expected Dashboard

ATM DASHBOARD	
Account Holder	: _____
Current Balance	: ₹ _____
Transaction Type	: _____
Transaction Amount	: ₹ _____
Sufficient Balance	: YES / NO
Within Limit	: YES / NO
Valid Amount	: YES / NO
Transaction Status	: APPROVED / DECLINED
Previous Balance	: ₹ _____
New Balance	: ₹ _____

3. Questions for Students

10. How will you calculate the new balance after withdrawal?
11. Which assignment operator can update the balance directly?
12. How will you check sufficient balance?
13. How will you check that the withdrawal amount is positive?
14. How can you check whether the transaction type is valid?
15. How will you combine all three withdrawal conditions?

4. Hints

- For withdrawal, balance -= amount.
- For deposit, balance += amount.
- Use >= to check sufficient balance.
- Use > 0 to ensure the amount is positive.
- Store valid transaction types in a list and use in.
- Use and for multiple compulsory conditions.
- Identify the Python operators used and explain why each was selected.

Case Study 5 — Smart University Access Dashboard

1. Problem Statement

A university has a smart access system. Different users have different access permissions. Authorized roles are Student, Faculty, and Admin. A user must have a valid ID card and an authorized role to enter a protected area.

2. Expected Dashboard

SMART ACCESS DASHBOARD	
User Name	: _____
Role	: _____
ID Card	: VALID / INVALID
Authorized Role	: YES / NO
ID Card Valid	: YES / NO
Restricted Area	: YES / NO
Access Level	: _____
ACCESS STATUS	: GRANTED / DENIED

3. Questions for Students

16. How can you check whether the user's role is in the authorized list?
17. Which operator checks membership?
18. How can you represent whether the ID card is valid?
19. What is the difference between `==` and `is`?
20. Can you demonstrate the difference using two Python objects?

4. Hints

- Create `authorized_roles = ['Student', 'Faculty', 'Admin']`.
- Think about role in `authorized_roles`.
- Use a Boolean variable such as `id_valid = True`.
- `==` compares values; `is` checks object identity.
- Try `a = [1,2,3]`, `b = a`, and `c = [1,2,3]`, then compare `==` and `is`.

Case Study 6 — Network Permission Dashboard

1. Problem Statement

A computer network stores user permissions using binary values. READ = 001, WRITE = 010, and EXECUTE = 100. A user may have one or more permissions.

2. Expected Dashboard

NETWORK PERMISSION DASHBOARD		
User Name	:	_____
User Role	:	_____
Permission	Binary	Status
READ	001	ALLOWED / DENIED
WRITE	010	ALLOWED / DENIED
EXECUTE	100	ALLOWED / DENIED
Permission Value	:	_____
Binary Value	:	_____

3. Questions for Students

21. How can you combine READ and WRITE permissions?
22. How can you check whether a user has READ permission?
23. What is the difference between and and &?
24. What does | do?
25. What does ^ do?
26. What is the purpose of << and >>?

4. Hints

- Start with READ = 1, WRITE = 2, EXECUTE = 4.
- Investigate READ | WRITE to combine permissions.
- Investigate permission & READ to check a permission.
- Convert numbers to binary to understand the results.
- Use shifts to explore movement of binary bits.

Case Study 7 — Smart University Cafeteria Dashboard

1. Problem Statement

A university cafeteria wants a Python application through which students can place food orders. The application should accept student information, display the menu, accept a food selection and quantity, calculate the bill, apply a student discount, validate a coupon, calculate delivery charges, check available balance, confirm or reject the order, update the balance, and display a digital receipt.

2. Expected Dashboard

SMART UNIVERSITY CAFETERIA	
STUDENT: Name _____	ID _____ Category _____
ORDER: Food Item _____	Quantity _____ Price ₹ _____
BILL	
Subtotal	: ₹ _____
Student Discount	: ₹ _____
Coupon Discount	: ₹ _____
Delivery Charge	: ₹ _____
FINAL AMOUNT	: ₹ _____
Menu Item	: ✓ AVAILABLE / X UNAVAILABLE
Coupon	: ✓ VALID / X INVALID
Balance	: ✓ SUFFICIENT / X INSUFFICIENT
ORDER CONFIRMED / REJECTED	

3. Questions for Students

27. How will you calculate the subtotal?
28. How will you determine whether the customer is eligible for a student discount?
29. How will you check whether the food item exists in the menu?
30. How will you validate the coupon?
31. How will you check whether the customer has sufficient balance?
32. How will you combine multiple conditions?
33. Which assignment operator can update the customer's balance?
34. Where can the ternary operator be used?
35. Can bitwise operators represent different cafeteria privileges?
36. Identify at least eight different Python operators used in your program.

4. Hints

- Create a menu list and use the membership operator.
- Subtotal = price * quantity.
- Use comparison to determine discount eligibility.
- Use logical operators when multiple conditions must be satisfied.
- After a successful purchase, balance -= final_amount.
- Use a ternary expression for CONFIRMED/REJECTED.
- Investigate bitwise | and & for privilege flags.
- Test operator precedence using price * quantity - discount + delivery.